see that the target evolves into something completely wrong. Let's now look at how to train the Dist-DQN.

Listing 7.10. Dist-DQN training on synthetic data

```
for i in range(1000):
    reward_batch = torch.Tensor([0,8]) + torch.randn(2)/10.0
1
    pred_batch = dist_dqn(state,theta,aspace=aspace)
2
    pred_batch2 = dist_dqn(state,theta_2,aspace=aspace)
3
    target_dist =
get_target_dist(pred_batch2,action_batch,reward_batch, \
                                    support, lim=
(vmin,vmax),gamma=gamma)     4
    loss = lossfn(pred_batch,target_dist.detach())
5
    losses.append(loss.item())
    loss.backward()
    # Gradient Descent
    with torch.no_grad():
        theta -= lr * theta.grad
    theta.requires_grad = True

    if i % update_rate == 0:
6
        theta_2 = theta.detach().clone()

fig,ax = plt.subplots(1,2)
ax[0].plot((target_dist.flatten(start_dim=1)
[0].data.numpy()),color='red',label='target')
ax[0].plot((pred_batch.flatten(start_dim=1)
[0].data.numpy()),color='green',label='pred')
ax[1].plot(losses)
```

- ***1*** Adds some random noise to the rewards to mitigate overfitting
- ***2*** Uses the main model Dist-DQN to make distribution predictions
- ***3*** Uses the target network Dist-DQN to make distribution predictions (using lagged parameters)
- ***4*** Uses the target network's distributions to create the target distribution for learning
- ***5*** Uses the main model's distribution prediction in the loss function
- ***6*** Synchronizes the target network parameters with the main model parameters